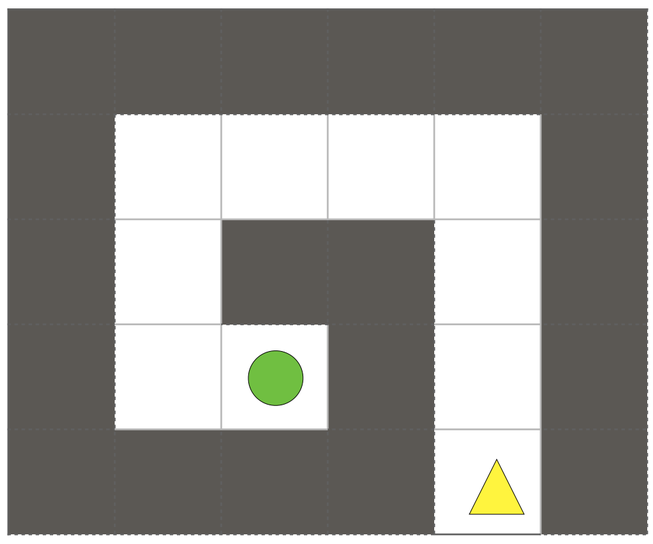
20. Computational Power

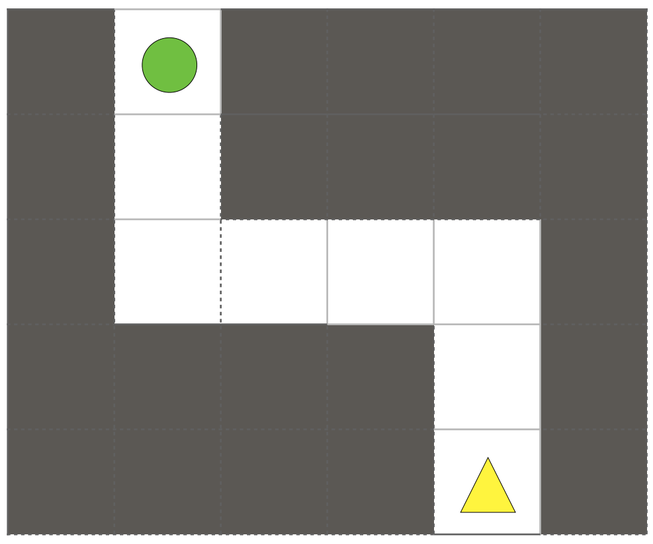
Differences in Computational Power: CCs vs SCs

Spiral



CC computable.
SC computable.

Zig-Zag



Not CC computable.
SC computable.

Differences in Power

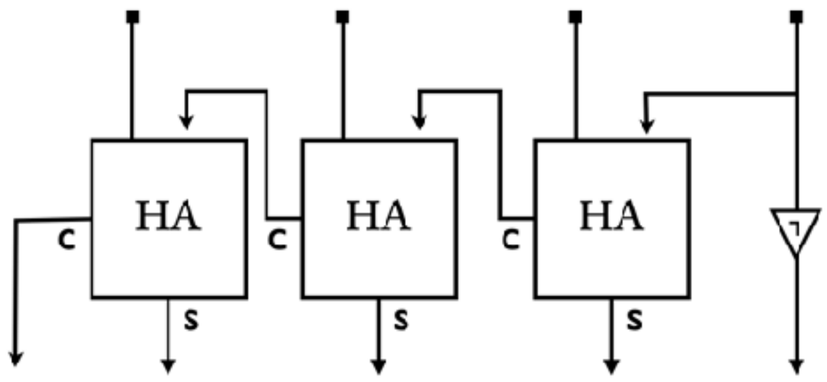
Zig-Zag is not CC computable because it requires different responses to the same input at different times. But CC's are history-independent..

Zig-Zag is SC computable, because SC memory allows the machine to enter into different input-output states over time, depending on its history.

The Memory Moral

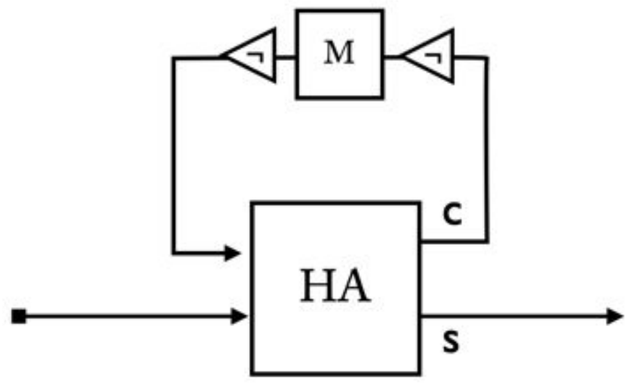
Memory allows a machine to display flexible responses to inputs. As a consequence, memory increases computational power.

Binary +1 [0-15]



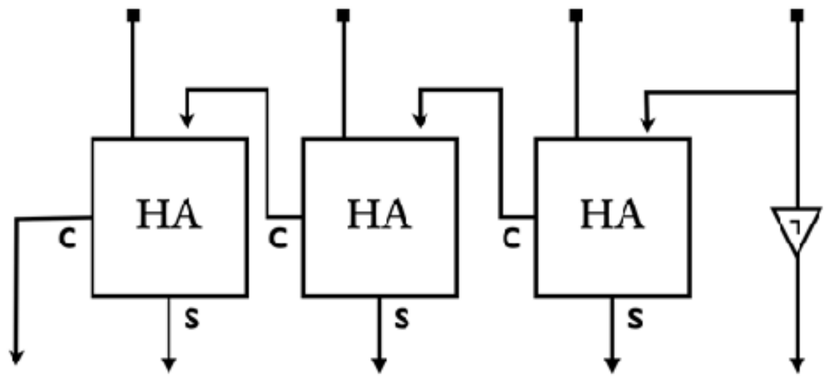
CC computable.
SC computable.

Binary +1



Not CC computable.
SC computable.

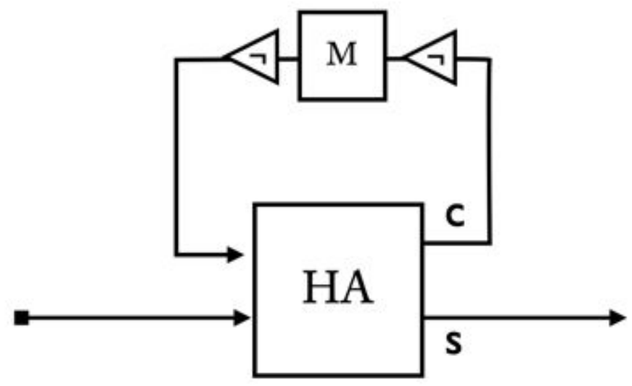
CC Powers and Limitations



CC Binary +1 [0-15]

- CC's have no memory or history responsiveness: they always respond in the same way to the same input.
- The input size for any given CC is fixed by its internal mechanisms.
- CC's can compute *any* bounded function.
- CC's compute *only* bounded functions.
- Practically: the higher the bound, the bigger the machine.

SC Powers and Limitations



SC Binary +1

- SC's contain memory and are history responsive: they can respond in different ways to the same input.
- They input size for any given SC is the infinite (or extendable) dimension of time.
- SC's can compute *any* bounded function.
- SC's compute *some* unbounded functions.
- Practically: for many functions, the higher the bound, the machine stays the same. (For other functions, this isn't true.)

"Infinite ends, finite means."

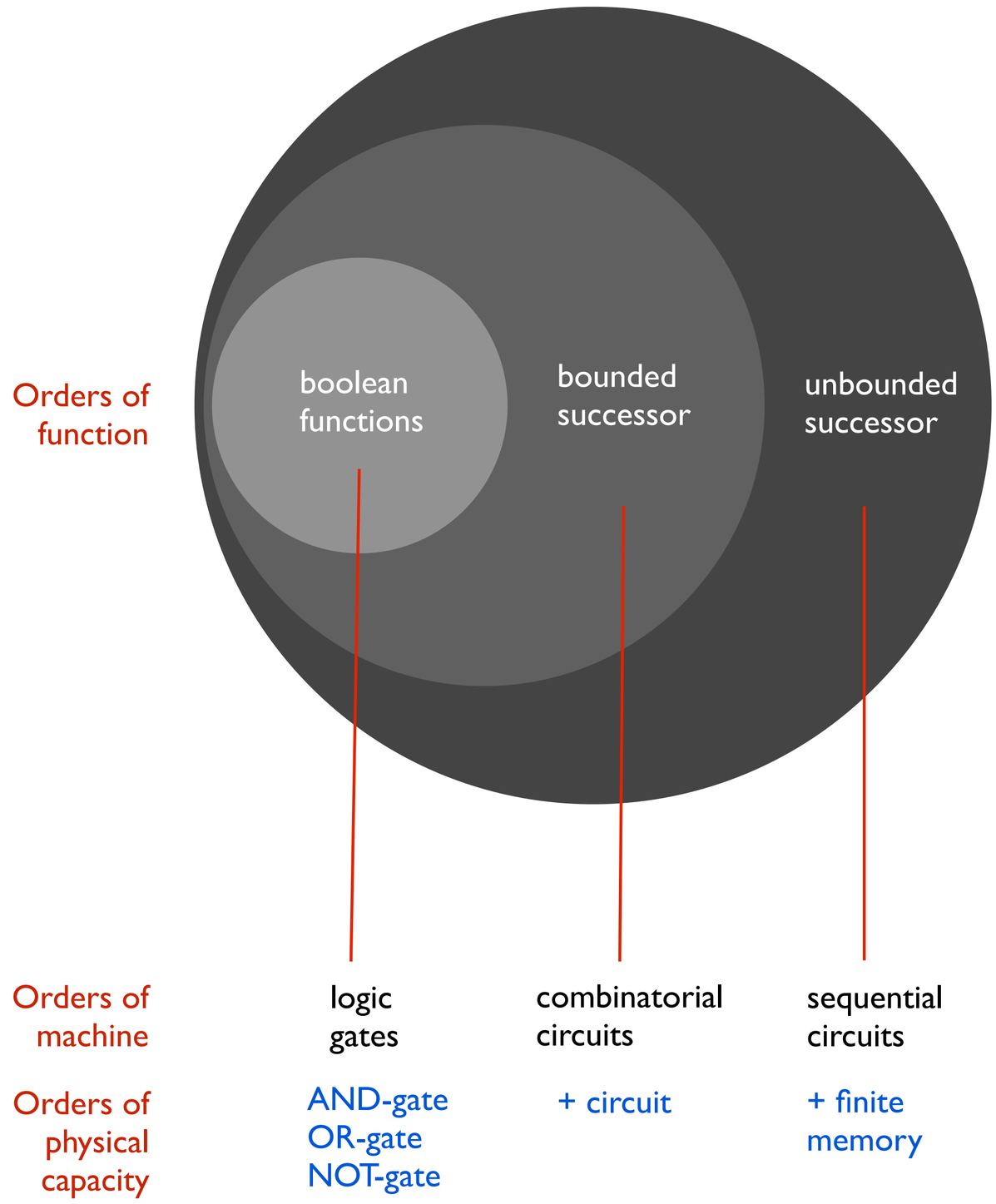
CC Implementation

- Fast.
- Limited in scope.
- The larger the problem, the larger the machine.
- Good for problems with fixed size.
- Compare: perception.

SC Implementation

- Slow.
- Unlimited in scope.
- The larger the problem, the machine stays the same.
- Good for problems with unpredictable size.
- Compare: thought.

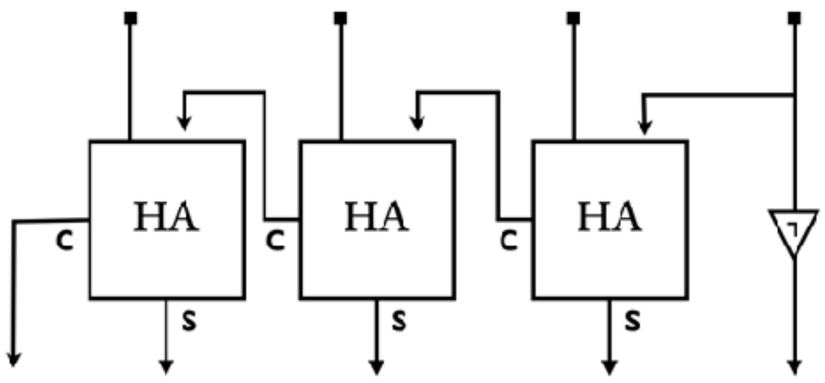
Orders of Computational Power



Definition: Computational Power
A machine M1 has greater computational power than a machine M2 iff

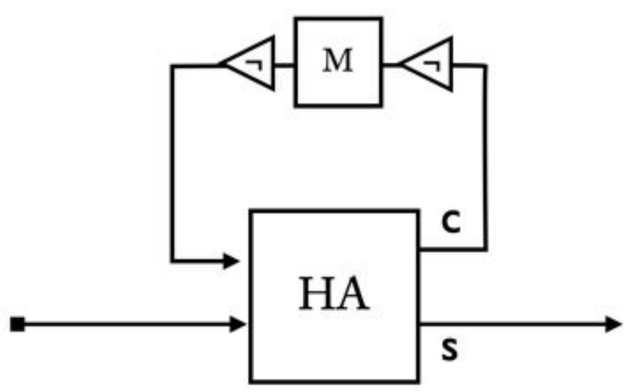
(a) for any function *f* and any code *C*:
if M1 can compute *f* under C, then M2 can compute *f* under C

(b) there some function *f* and code *C* such that:
M1 can compute *f* under *C*, but *M2* cannot compute *f* under *C*.



Binary +1 [0-15]

CC computable.
SC computable.

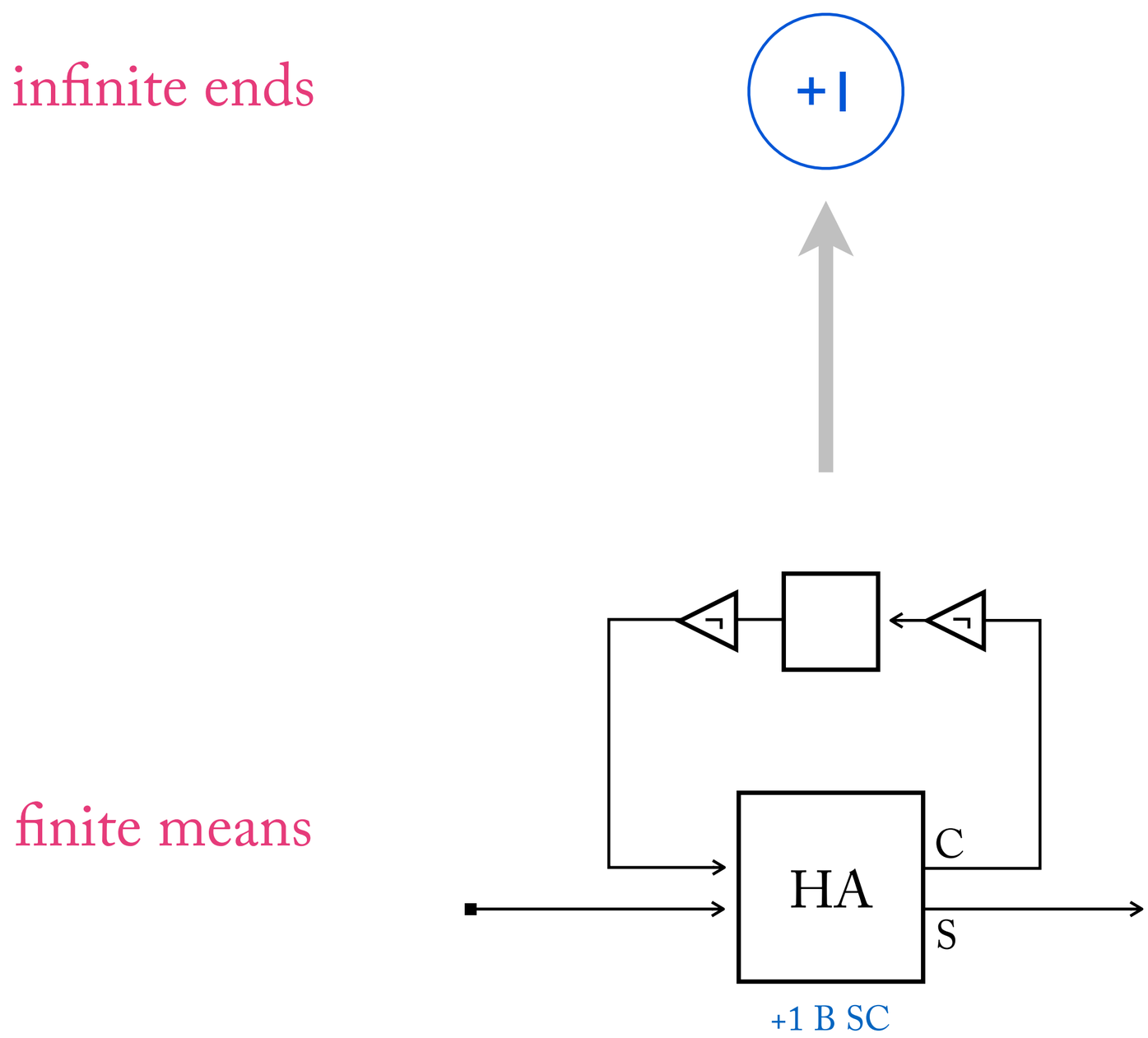


Binary +1

Not CC computable.
SC computable.

For language is quite peculiarly confronted by an an unending and truly boundless domain, the essence of all that can be thought. It must therefore make infinite employment of finite means, and is able to do so through the power who produces identity of language and thought.

— Humbolt, *On Language*, 1836



What allows SC's to overcome their own finite physical limitations? They exploit the potential infinity of time.

Memory isn't just necessary for achieving infinite potential. It seems to be necessary for *coherence* too.

Language. I'd like to buy a carne asada taco.

Argument. I think, therefore I am.

A Map of Power and Abstraction

Different classes of function are computed by machines with different physical features. The more functions computable by a kind of machine, the greater its *computational power*.

